UNIVERSITY OF WESTMINSTER

INFORMATICS INSTITUTE OF TECHNOLOGY

Client-Server Architectures

5COSC022C.2

Coursework

Student Name – Mohamed Aqeel Jameel

UoW Number – w2120306

IIT Number – 20231468

ACKNOWLEDGMENT

I wish to extend my heartfelt thanks to Mr. Cassim Farook for his exceptional guidance and the rich insights he shared throughout this module. His mentorship has been a profound source of inspiration. I am equally grateful to my parents for their unwavering encouragement and steadfast support, which have been essential to my journey. Additionally, I would like to acknowledge the collaborative spirit and assistance of my colleagues, whose contributions were instrumental in the successful completion of this report.

TABLE OF CONTENTS

ACKNOWLEDGMENT i
Smart Campus Sensor & Room 1
Conceptual Report (Question & Answer) 1
Part 1.1. JAX-RS resource lifecycle & concurrency 1
Part 1.2. Why HATEOAS matters 1
Part 2.1. IDs vs full objects in list responses 1
Part 2.2. Is DELETE idempotent here? 2
Part 3.1. @Consumes(APPLICATION_JSON) mismatch 2
Part 3.2. @QueryParam vs path for filtering 2
Part 4.1. Benefits of the sub-resource locator pattern 3
Part 5.2. Why 422, not 404 3
Part 5.4. Cybersecurity risk of leaked stack traces 3
Part 5.5. Why filters for cross-cutting concerns 4

Smart Campus Sensor & Room

Conceptual Report (Question & Answer)

Part 1.1. JAX-RS resource lifecycle & concurrency

By default, JAX-RS treats a root resource class as per-request: a fresh instance is constructed for every incoming HTTP request, so instance fields cannot be assumed to survive across requests. Any state that must persist (the rooms, sensors, and readings maps in this API) therefore has to live outside the resource, here it lives in the DataStore enum singleton, which exists for the lifetime of the JVM. Because Grizzly dispatches requests on a worker-thread pool, that singleton is accessed concurrently; I used ConcurrentHashMap for the top-level collections and Collections.synchronizedList for the per-sensor reading history so that map reads, map writes, and list appends are all safe without coarse-grained synchronized blocks. A plain HashMap risks losing entries or throwing ConcurrentModificationException during a concurrent re-hash; caching data on resource fields would effectively throw it away every request.

Part 1.2. Why HATEOAS matters

The /api/v1 discovery endpoint returns not only metadata but a resources map whose values are absolute URIs to the top-level collections, with supported verbs and nested sub-resource links. This is the HATEOAS idea: the server, not the client, is the source of truth for what actions are possible and where. A client that started at /api/v1 and followed links never needs to hard-code a URL pattern; if the API later moves sensors to a new path or publishes a new resource, the client picks it up automatically. Compared to static documentation, hypermedia (1) couples the client to URI roles instead of URI shapes, (2) lets the server progressively enable/disable capabilities at runtime, and (3) makes the API discoverable from the root, which is how browsers have scaled the human web.

Part 2.1. IDs vs full objects in list responses

GET /rooms currently returns full Room objects. Returning only IDs would shrink each response dramatically and save bandwidth when the caller just needs to page through identifiers (e.g. an autocomplete). The cost is that every client that actually needs name or capacity must make a follow-up GET /rooms/{id} for every single entry - the classic “N+1 request” problem, which is usually slower overall than one larger payload. The pragmatic middle ground is a thin summary DTO (id, name, capacity) on the collection endpoint and the full object on the detail endpoint; since the coursework’s Room already is small, returning the full objects is a reasonable default.

Part 2.2. Is DELETE idempotent here?

Yes. Idempotency is a statement about resulting state, not the response code. First DELETE /rooms/LAB-101 returns 204 and removes the room; a second identical call returns 404 (the room is already gone), but the state of the server is unchanged - it is still “no LAB-101”. Because the end state is the same no matter how many times the call is made, the operationsatisfies RFC 7231’s definition of idempotent. Note this is distinct from “safe”: DELETE does mutate state, so it is idempotent but not safe.

Part 3.1. @Consumes(APPLICATION_JSON) mismatch

@Consumes is used during content negotiation, before the resource method is invoked. When a client sends Content-Type: text/plain to an endpoint that only declares @Consumes(MediaType.APPLICATION_JSON), Jersey’s request matcher cannot find a method able to accept that body, so the framework short-circuits and returns 415 Unsupported Media Type. The method body never runs, which is exactly what we want MessageBodyReader<Sensor> for JSON would never have been able to parse a plain-text payload, and letting it try would surface an internal parsing error to the client instead of a well-defined protocol-level response.

Part 3.2. @QueryParam vs path for filtering

A query parameter (/sensors?type=CO2) expresses “the same collection, narrowed by a criterion”. A path segment (/sensors/type/CO2) would assert that CO2 is a distinct sub-resource, which it isn’t, it’s the same collection filtered differently. Query parameters also compose cleanly (?type=CO &status=ACTIVE&limit=50) while path segments force a combinatorial explosion of routes. Caching behavior is also clearer: intermediaries treat ?type=CO2 as a variant of the parent resource rather than as a separate entity, which matches the real semantics.

Part 4.1. Benefits of the sub-resource locator pattern

SensorResource.readings(...) is a sub-resource locator: it has no HTTP verb annotation, it only resolves the path prefix and returns a SensorReadingResource instance. That handoff gives us two concrete benefits. First, separation of concerns, SensorResource does not need to know anything about reading history, and SensorReadingResource only knows about one concrete sensor (it receives it in its constructor, so every method inside has trivial access to parentSensor). Second, composability - when tomorrow we add filtering or aggregation on readings, the new @GET / @POST methods simply live in SensorReadingResource instead of bloating SensorResource with sensors/{id}/readings/... endpoints. The alternative, a single monolithic controller handling every path level, scales linearly with complexity and becomes unreadable quickly.

Part 5.2. Why 422, not 404

A 404 means “the URL you asked for does not name anything in my universe.” That is not what happened when the client POSTs {"roomId":"NOPE"} to /sensors: the URL /sensors is valid, the method is valid, and the JSON parses correctly - the body just references a room that does not exist. 422 Unprocessable Entity was introduced by WebDAV and reused by modern APIs for exactly this scenario: syntactically valid, semantically invalid. Returning 404 would confuse the client into thinking /sensors itself is gone, which would break retry logic or monitoring that watches for missing endpoints. 422 keeps the diagnosis inside the payload, where the field and value pinpoint the exact problem.

Part 5.4. Cybersecurity risk of leaked stack traces

GenericExceptionMapper logs the full Throwable to the server but returns a bland JSON body to the client. Returning the raw trace instead would hand attackers a reconnaissance goldmine: the exact Jersey version (useful for CVE matching), the JDK build, internal package names that reveal project layout, file paths that hint at the deployment environment, and sometimes snippets of query text or sanitized SQL. An attacker with that information can target known vulnerabilities, phrase social-engineering attacks more convincingly, or probe more efficiently for misconfigurations. The OWASP “Improper Error Handling” category exists because of exactly this failure mode.

Part 5.5. Why filters for cross-cutting concerns

LoggingFilter is one class that logs every request and every response for the entire API. If the same observability were expressed as Logger.info(...) calls inside each resource method, you would need to remember to add them to every new endpoint, the log format would drift across contributors, and refactoring would touch dozens of files to change one field. Filters make logging pluggable - annotate with @Provider, register once, and every request/response flows through it, including error responses produced by exception mappers. The same mechanism scales to authentication, request-id propagation, metrics, and CORS, which is why JAX-RS ships filters as the supported extension point for cross-cutting behaviour.